

Cahier d'Atelier

MVP Data Lakehouse Contract-Driven — Aide sociale cantonale

A. EL HAJJAJI

Atelier de trois journées · 35 règles de gestion pilotées par contrat de données · plateforme complète sous Docker Compose (avec Airflow) · tout le code de ce cahier est exécutable depuis ce repository et vérifié par la CI

Cas d'usage : ce cas d'usage cible l'analyse des données sociales — versements d'aide sociale et d'aide aux migrants — pour permettre à l'institution qui les détient d'en assurer le pilotage, la conformité et l'audit financier.

Dataset source : PostgreSQL `pocs` — schéma `src` — 4 tables : `individu`, `dossier`, `permis`, `prestation` (repository `elhajjaji/benefits-dataset`). **La base source est déjà disponible** (conteneurs `pocs-postgres` et `pocs-pgadmin`) : l'atelier ne la construit pas, il la consomme.

Stack technique : PostgreSQL 15+, Apache NiFi, Redpanda + Debezium (CDC), MinIO (S3), Apache Iceberg, Project Nessie, **Apache Airflow**, dbt Core 1.8+ (dbt-utils, dbt-expectations, unit tests, Elementary), OpenLineage/Marquez, Dremio, Metabase, GitHub Actions, SQLFluff, pre-commit.

Public et prérequis : Data Engineers, Business Analysts et Tech Leads. Prérequis : SQL courant, notions de Git (branche, PR), Docker installé (16 Go de RAM recommandés — les profils Compose permettent de travailler avec moins).

Principe pédagogique du cahier : chaque séquence commence par 🎯 **l'objectif** (ce qu'on saura faire à la fin) et 🧰 **les outils** (à quoi sert chaque brique et *pourquoi* elle est là), *avant* de toucher au code. Les encadrés 📖 **Concept** posent la théorie au moment où elle sert. Chaque séquence se termine par un ✅ **checkpoint** que l'animateur valide avec le groupe avant de passer à la suite.

Programme des trois journées

Chaque journée représente environ 7 h de face-à-face (pauses et déjeuner non compris). Les durées sont indicatives : l'animateur adapte au rythme du groupe, en préservant la place du mini-projet final.

Jour 1 — Fondations : le métier, le contrat, la plateforme, le Bronze

Séquence	Durée
S1 — Comprendre le métier avant de coder	1 h 30
S2 — Les 35 règles de gestion et l'approche contract-driven	1 h 45
S3 — Démarrer la plateforme	1 h 30
S4 — Zone Bronze : ingestion, traçabilité et qualité technique	2 h
Débrief du jour, questions	15 min

Jour 2 — Transformer et prouver : Silver, Gold, tests

Séquence	Durée
Rappels J1 (quiz express)	15 min
S5 — Zone Silver : le contrat à l'œuvre	2 h 30
S6 — Zone Gold : modélisation et RG métier avancées	2 h
S7 — La pyramide de tests et le cycle WAP	2 h
Débrief du jour	15 min

Jour 3 — Industrialiser : CI/CD, orchestration, lignage, restitution, mini-projet

Séquence	Durée
S8 — CI/CD : du poste du développeur à la production	1 h 30
S9 — Orchestration avec Airflow	1 h
S10 — Lignage, documentation-as-code et outillage DPO	1 h 15
S11 — Restitution : Dremio et Metabase	45 min
S12 — Mini-projet fil rouge : votre RG_36 de bout en bout	2 h 30
Synthèse, restitution des mini-projets, quiz de clôture	30 min

JOUR 1 — Fondations

S1 — Comprendre le métier avant de coder

 **Objectif** : savoir expliquer, sans jargon technique, ce qu'est un dossier familial, qui sont les deux populations bénéficiaires, à quoi sert chaque table source — et savoir *interroger le métier* : la moitié des 35 RG de cet atelier sont nées de questions posées à des gestionnaires de dossiers, pas d'une spécification.

 **Outils** : un paperboard, et **pgAdmin** (<http://localhost:5050>) pour l'exercice d'exploration. C'est volontaire : la première heure d'un projet data réussi ne se passe pas dans un IDE.

1.1. Le cas d'usage

Ce cas d'usage cible l'analyse des données sociales pour permettre à l'institution qui les détient de piloter, contrôler et auditer ses versements. Cette institution assume un mandat de service social cantonal : venir en aide aux personnes les plus démunies, résidents du canton en difficulté financière comme ressortissants étrangers (demandeurs d'asile, réfugiés,

personnes admises provisoirement). Elle verse des **prestations financières** directement aux individus et les accompagne socialement. Le dataset représente ces versements et les structures administratives qui les encadrent.

Trois acteurs traversent tout l'atelier — leur point de vue justifie les règles :

Acteur	Ce qui l'inquiète	RG qui le protègent
Le gestionnaire de dossier	verser juste, à la bonne personne, une seule fois	RG_13, 16, 17, 26
Le contrôleur financier	les montants, les plafonds, les cumuls	RG_14, 15, 19, 24
Le DPO (protection des données)	qui voit quoi, combien de temps, avec quelle trace	RG_07, 28, 33, 34, 35

1.2. Les quatre tables sources (`pocs`, schéma `src`)

Table	Grain	Colonnes	Rôle métier
<code>src.individu</code>	1 ligne = 1 bénéficiaire	<code>id</code> , <code>numero_individu</code> , <code>nom</code> , <code>prenom</code> , <code>email</code> , <code>navs</code> , <code>sexe</code> , <code>date_naissance</code> , <code>date_deces</code>	Le référentiel des personnes. Contient des données personnelles (nom, email, NAVS) → obligations LPD/RGPD.
<code>src.dossier</code>	1 ligne = 1 rattachement individu ↔ dossier, avec période	<code>id</code> , <code>numero_dossier</code> , <code>numero_individu</code> , <code>date_debut</code> , <code>date_fin</code>	L'unité de prise en charge familiale . <code>date_fin</code> NULL = rattachement encore actif.
<code>src.permis</code>	1 ligne = 1 permis de séjour, avec période	<code>id</code> , <code>numero_individu</code> , <code>permis</code> , <code>date_debut</code> , <code>date_fin</code>	Le statut légal de séjour des migrants (référentiel OCPM : B, C, F, N, S, L).
<code>src.prestation</code>	1 ligne = 1 versement	<code>id</code> , <code>numero_individu</code> , <code>type_prestation</code> , <code>montant_prestation</code> , <code>date_debut_prestation</code> , <code>date_fin_prestation</code> , <code>date_prestation</code>	Le cœur financier : chaque versement, son type, son montant, sa période de couverture et sa date de paiement.

 **Concept — les trois dates d'une prestation.** `date_debut_prestation` et `date_fin_prestation` bornent la période *couverte* (ex. le loyer de janvier) ; `date_prestation` est la date du *paiement*. Les contrôles de couverture (RG_18) et d'âge (RG_25) s'évaluent à la **date du paiement** — confondre ces dates est l'erreur d'analyse la plus fréquente sur ce dataset.

1.3. La notion de dossier familial

C'est le concept le plus important des trois jours. Un dossier n'est pas individuel — il regroupe tous les membres d'une même famille vivant sous le même toit, qui partagent le même `numero_dossier` :

DOS-2020-001

- IND-001 : Jean Dupont (père) → rattaché depuis 01/01/2020
- IND-002 : Marie Dupont (mère) → rattaché depuis 01/01/2020
- IND-003 : Lucas Dupont (fils) → rattaché jusqu'au 30/06/2023
- IND-004 : Emma Dupont (fille) → rattaché depuis 01/01/2020

Quand Lucas quitte le foyer le 01/07/2023 tout en restant bénéficiaire, son rattachement à DOS-2020-001 est **clôturé** au 30/06/2023 et un **nouveau dossier** DOS-2023-042 est créé pour lui seul. Il ne peut jamais être dans deux dossiers actifs en même temps (ce sera la RG_13).

Trois conséquences directes sur les règles de gestion :

- 1. Les **prestations sont versées à l'individu** — sauf le LOYER, versé une seule fois par foyer (dossier) et par mois (RG_17).
- 2. La **vérification de couverture** (RG_18) regarde si l'individu avait un dossier *ou* un permis actif à la date du versement.
- 3. La **détection des doublons de loyer** se fait au grain du dossier, jamais de l'individu.

1.4. Les deux populations bénéficiaires

Population	Couverture	Prestations principales
Aide sociale (résidents du canton)	Dossier familial uniquement	FORFAIT_ENTRETIEN, LOYER, PRIME_LAMAL, AVANCE_RENTE

Population	Couverture	Prestations principales
Migrants (ressortissants étrangers)	Dossier familial et permis de séjour	FORFAIT_ENTRETIEN, FORFAIT_INTEGRATION, AIDE_URGENCE, FORFAIT_ETSP

Les migrants ont toujours un dossier ET un permis. Conséquence en creux, découverte en interviewant le métier : **une prestation « migrant » versée à quelqu'un qui n'a pas de permis actif est suspecte** — ce sera la RG_27.

1.5. Les types de prestations (LASLP)

Type	Description	Fréquence	Plafond
FORFAIT_ENTRETIEN	Forfait mensuel (alimentation, vêtements, énergie...)	1 fois/individu/mois	1 800 CHF
FORFAIT_INTEGRATION	Aide à l'intégration sociale et linguistique	Variable	—
LOYER	Prise en charge du loyer du foyer	1 fois/ dossier /mois	2 500 CHF
PRIME_LAMAL	Aide aux primes assurance-maladie	Mensuel, 18-25 ans uniquement	—
AIDE_URGENCE	Forfait quotidien d'urgence (déboutés, NEM)	Ponctuel, plusieurs fois par mois OK	100 CHF/jour
AVANCE_RENTE	Avance sur rente AVS/AI en attente	Ponctuel	3 500 CHF
AIDE_EXCEPTIONNELLE	Aide barème 2 pour situations complexes (ETSP)	Variable	—
AIDE_MATERIELLE	Achat de matériel ou d'équipement	Ponctuel	—
FORFAIT_ETSP	Forfait hébergement particulier ou troubles importants	Mensuel	—

⚠ Deux pièges métier : les seniors (≥ 65 ans) relèvent des prestations complémentaires AVS/AI — un FORFAIT_ENTRETIEN versé à un senior est une anomalie (RG_20) ; et la PRIME_LAMAL versée hors de la tranche 18-25 ans en est une autre (RG_25). Dans les deux cas **l'âge s'évalue à la date du versement**.

1.6. Exercice — explorer la source avec pgAdmin (20 min)

Ouvrir pgAdmin (<http://localhost:5050>), naviguer jusqu'au schéma `src`, puis répondre en SQL :

- Combien de membres compte le dossier le plus nombreux ?
- Y a-t-il des individus présents dans `prestation` mais absents d'`individu` ? (*spoiler : ce sera RG_23*)
- Trouvez un individu avec deux permis dont les périodes se chevauchent. (RG_29)
- Quelle question poseriez-vous au métier sur ce que vous venez de voir ? — chaque binôme en formule une, l'animateur les collecte : plusieurs correspondront à des RG du chapitre suivant, et les meilleures serviront de sujet libre au mini-projet (S12).

✅ **Checkpoint S1** : chaque participant reformule la différence entre « prestation versée à l'individu » et « loyer versé au dossier », explique pourquoi Lucas a deux lignes dans `src.dossier`, et cite les trois dates d'une prestation sans se tromper.

S2 — Les 35 règles de gestion et l'approche contract-driven

🎯 **Objectif** : disposer d'un langage commun BA ↔ DE ↔ DPO : la liste des 35 RG, leur gravité, et surtout **où vit chaque règle** (contrat déclaratif, macro algorithmique, hook de conformité). C'est la carte qu'on suivra pendant trois jours.

🧰 **Outils** :

- **Le contrat de données** — bloc `vars.contracts` de `dbt/dbt_project.yml` : la source de vérité *machine-readable* des règles déclaratives (domaines de valeurs, plafonds, seuils, regex, volumétries minimales). Éditable par le BA, en YAML pur, sans SQL.
- **Les contrats métier** — `dbt/contracts/` : un fichier YAML par entité qui documente le *pourquoi* de chaque règle, ses exceptions connues et sa gouvernance.
- **Les macros dbt** — `dbt/macros/` : le *comment* des règles algorithmiques, celles qu'un YAML ne peut pas exprimer.
- **Les hooks et opérations dbt** — pour les règles de *conformité* qui gouvernent l'exécution elle-même (journalisation RG_34, droit à l'oubli RG_35).

2.1. 📖 Concept — qu'est-ce qu'un data contract ?

Un data contract est un accord **versionné, revu et exécutable** entre producteurs et consommateurs de données. Il répond à quatre questions : quelle *forme* (schéma, types), quel *fond* (domaines, bornes, invariants), quelle *fraîcheur*, quelle *gouvernance* (qui valide un changement). La différence avec une spécification Word : le contrat est **dans le repository**, la CI le fait respecter, et le diff Git raconte son histoire. Dans ce projet, le contrat a trois étages :

Étage	Fichier	Question	Qui édite
Contrat de schéma	<code>models/marts/schema.yml</code> (<code>contract: enforced</code>)	la <i>forme</i> : colonnes et types de <code>fct_prestations</code>	DE
Contrat de données	<code>dbt_project.yml</code> (<code>vars.contracts</code>)	le <i>fond</i> : domaines, plafonds, seuils, regex	BA (via PR)
Contrat métier	<code>contracts/*.yaml</code>	le <i>pourquoi</i> : intention, exceptions, propriétaires	BA

2.2. Le principe : un hybride discipliné

Tout mettre dans le contrat serait illusoire : comment déclarer « deux périodes ne se chevauchent pas » en YAML sans réinventer un langage ? Tout mettre en macros serait opaque pour le métier. La ligne de partage :

Nature de la règle	Où elle vit	Exemple
Déclarative (liste, seuil, plafond, regex)	<code>vars.contracts</code> — les macros et tests la lisent , jamais de valeur en dur dans le SQL	Domaine des permis, plafond LOYER, tranche d'âge LAMAL, format NAVS
Algorithmique (jointures, fenêtres, self-join)	Macros + tests singuliers — le contrat métier n'en documente que l' <i>intention</i>	Chevauchement de périodes, réconciliation volumétrique
De conformité (gouverne l'exécution)	Hooks <code>on-run-end</code> et <code>run-operation</code>	Journal des accès PII, droit à l'oubli

Concrètement : pour ajouter un 10^e type de prestation, changer un plafond ou élargir la tranche LAMAL, le BA modifie **une ligne de YAML**, ouvre une PR, et la CI revalide tout. C'est la démonstration centrale des trois jours.

2.3. Les 35 règles

Zone Bronze — traçabilité, fraîcheur et qualité technique

RG	Nom	Résumé	Gravité
RG_01	Traçabilité d'ingestion	Chaque ligne porte <code>_ingested_at</code> , <code>_source_system</code> , <code>_batch_id</code>	Obligatoire
RG_02	Fraîcheur des sources	Alerte à 24 h sans nouvelle donnée, erreur à 72 h	Alerte → Bloquant
RG_30	Volumétrie minimale	Une table Bronze anormalement vide bloque le run (seuils au contrat)	Bloquant
RG_31	Anomalies statistiques de volume	Elementary apprend le volume habituel et alerte sur l'écart	Alerte

RG	Nom	Résumé	Gravité
RG_32	Dérive de schéma source	Colonne apparue/disparue/retypée dans la source = alerte	Alerte

Zone Silver — qualité et standardisation

RG	Nom	Résumé	Gravité	Vit dans
RG_03	Déduplication technique	Garder la version la plus récente (double ingestion NiFi/Redpanda)	Bloquant	macro
RG_04	Clé surrogate déterministe	Hash stable pour toutes les entités	Obligatoire	macro
RG_05	Normalisation du sexe	H/F/X/vide → libellés normalisés	Informatif	modèle
RG_06	Cohérence biographique	1900 ≤ naissance ≤ aujourd'hui ; décès ≥ naissance	Bloquant	modèle + contrat
RG_07	Anonymisation PII	nom, prénom, email, NAVS masqués selon le rôle	Bloquant	modèle + vars
RG_08	Périodes ouvertes	date_fin NULL → 9999-12-31 pour fiabiliser les BETWEEN	Obligatoire	macro
RG_09	Domaine permis OCPM	Seuls B, C, F, N, S, L sont valides	Bloquant	contrat
RG_10	Domaine type prestation	Seuls les 9 types LASLP sont valides	Bloquant	contrat
RG_11	Historisation SCD2 permis	Chaque changement de statut (N→F→B...) est conservé	Obligatoire	snapshot
RG_12	Union de couverture	Dossiers + permis valides = une seule table de couverture	Obligatoire	modèle
RG_13	Non-chevauchement dossiers	Un individu jamais dans deux dossiers actifs simultanément	Bloquant	macro
RG_28	Format NAVS13	Le numéro AVS respecte <code>756.xxxx.xxxx.xx</code> (regex au contrat)	Bloquant	contrat

Zone Gold — règles métier avancées

RG	Nom	Résumé	Gravité	Vit dans
RG_14	Montant strictement positif	Montant ≤ 0 = fail-fast	Bloquant	test
RG_15	Plausibilité montant	Montant au-dessus du barème institutionnel = alerte	Alerte	contrat (plafonds)
RG_16	Unicité forfait mensuel	Un seul FORFAIT_ENTRETIEN par individu et par mois	Bloquant	test singulier
RG_17	Unicité LOYER par dossier	Un seul LOYER par dossier et par mois	Bloquant	modèle + test
RG_18	Conformité couverture	Versement adossé à un dossier/permis actif à sa date	Alerte	modèle
RG_19	Plafond cumul mensuel	Cumul mensuel individu > 5 000 CHF = alerte DPO	Alerte	contrat (seuil)
RG_20	Senior + forfait entretien	≥ 65 ans + FORFAIT_ENTRETIEN = anomalie	Alerte	contrat (âge)
RG_21	Dimension calendrier	Axe temps continu, jour par jour, depuis 2020	Obligatoire	modèle
RG_22	Coût journalier sécurisé	Montant / durée en jours, division par zéro impossible	Obligatoire	modèle + unit test
RG_25	Âge PRIME_LAMAL	Réservée aux 18-25 ans, à la date du versement	Alerte	contrat (bornes)
RG_26	Versement après décès	date_prestation > date_deces = anomalie grave	Bloquant	modèle + test
RG_27	Migrant sans permis	Types migrants sans permis valide actif = alerte	Alerte	contrat (types)

Couches transverses — intégrité, industrialisation, conformité

RG	Nom	Résumé	Gravité
RG_23	Intégrité référentielle	Aucune prestation liée à un individu inconnu	Bloquant
RG_24	Réconciliation volumétrique	Moins de 10 % de perte entre Bronze et Gold	Bloquant
RG_29	Non-chevauchement permis	Un seul statut légal de séjour à la fois (même macro que RG_13)	Bloquant
RG_33	Minimisation PII en Gold	Aucune colonne nominative en zone Gold (contrôle du <i>catalogue</i>)	Bloquant
RG_34	Journal des accès PII	Chaque run tracé : rôle, exposition PII, horodatage	Obligatoire
RG_35	Droit à l'oubli	Purge d'un individu sur demande, journalisée, propagée au build	Obligatoire

2.4. Gravités : trois comportements distincts

- **Bloquant (fail-fast)** : en mode strict, le test échoue et **stoppe le pipeline** — la donnée fausse ne se propage pas.
- **Alerte** : le pipeline continue, l'anomalie est **flaguée** dans une colonne `is_*` et remonte dans les dashboards et le rapport d'audit DPO. On choisit l'alerte quand une dérogation métier est possible (RG_15) ou quand bloquer ferait plus de mal que l'anomalie (RG_18).
- **Obligatoire / Informatif** : transformation ou mécanisme toujours appliqué, sans test d'échec.

💡 **Astuce d'atelier** : la variable `rg_severity` (défaut `warn`) permet de dérouler les trois jours sans casse, puis de basculer en fail-fast d'une seule commande (`make test-strict`). En production, elle vaut `error`. Les RG « alerte » restent `warn` même en mode strict — c'est un choix métier, pas technique.

2.5. Exercice — qualifier des règles (20 min)

Par binôme, pour chacune des situations suivantes, décider : déclaratif / algorithmique / conformité ? bloquant / alerte ? puis vérifier dans le code où elle vit réellement :

1. « Le plafond du LOYER passe à 2 800 CHF au 1er janvier. »
2. « Personne ne doit toucher deux loyers le même mois dans le même foyer. »
3. « Le DPO veut savoir qui a lancé des builds avec les PII en clair. »
4. « Un NAVS doit commencer par 756. »
5. « On perd 40 % des lignes entre Bronze et Gold depuis hier. »

✅ **Checkpoint S2** : ouvrir [dbt/dbt_project.yml](#) et [dbt/contracts/prestation.yml](#) côte à côte ; le groupe sait dire pour chaque étage du contrat *qui* l'édite et *ce qui* le fait respecter.

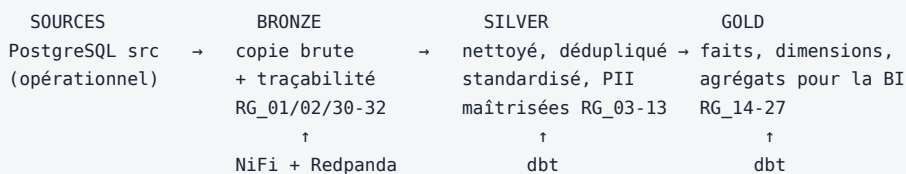
S3 — Démarrer la plateforme

🎯 **Objectif** : une plateforme lakehouse complète qui tourne sur le poste de chacun, la capacité de vérifier que chaque brique est en vie, et une carte mentale claire : *quel conteneur joue quel rôle dans quelle séquence*.

🧰 **Outils** :

- **Docker Compose** — décrit les ~13 conteneurs dans `docker-compose.yml`, avec des **profils** pour démarrer seulement ce dont on a besoin.
- **PostgreSQL** — la base source `pocs` (**déjà disponible** : conteneurs `pocs-postgres`, port 5433, et `pocs-pgadmin`, port 5050). Elle héberge aussi, dans l'atelier, les schémas `bronze` / `silver` / `gold` / `audit` que dbt construit.
- **MinIO** — le stockage objet compatible S3 : le « disque dur » du lakehouse.
- **Nessie** — le catalogue Iceberg **versionné comme Git** : branches, merges, Zero-Copy. Indispensable au cycle WAP (S7).
- **NiFi / Redpanda / Debezium** — l'ingestion (S4).
- **Airflow** — l'orchestrateur : qui lance quoi, quand, et que se passe-t-il en cas d'échec (S9).
- **Dremio / Metabase** — la restitution (S11).
- **Marquez** — le serveur de lignage OpenLineage (S10).

3.1. 📖 Concept — l'architecture médaillon



Pourquoi trois zones ? **Bronze** garde la vérité brute (rejouable, auditable) ; **Silver** porte la qualité (une seule fois, pour tous les usages) ; **Gold** porte le sens métier (modèles en étoile, agrégats). Une règle de gestion s'applique **le plus tôt possible mais pas plus tôt** : la déduplication en Silver, le plafond de cumul en Gold — jamais l'inverse.

3.2. Démarrage

📌 **La base source tourne déjà** (`pocs-postgres` / `pocs-pgadmin`). Le `.env.example` pointe dessus par défaut ; le service `postgres` du compose n'est qu'un secours.

```
cp .env.example .env
docker compose --profile "*" up -d      # tout démarrer (~ 2-3 min, hors Airflow)
docker compose --profile "*" ps        # tout doit être Up / healthy
```

Sur une machine à moins de 16 Go de RAM, démarrer par étage au fil des jours :

```
docker compose --profile lakehouse --profile ingestion up -d  # Jour 1
docker compose --profile orchestration up -d                  # Jour 3 matin
docker compose --profile bi --profile lineage up -d           # Jour 3
```

3.3. Tour du propriétaire

Service	URL	Vérification
PostgreSQL source	<code>localhost:5433</code>	<code>docker exec -it pocs-postgres psql -U user -d pocs -c '\dt src.*'</code>
pgAdmin	<code>http://localhost:5050</code>	naviguer jusqu'au schéma <code>src</code>
MinIO Console	<code>http://localhost:9001</code>	le bucket <code>warehouse</code> existe
Nessie	<code>http://localhost:19120</code>	l'API répond, la branche <code>main</code> existe

Service	URL	Vérification
NiFi	https://localhost:8443/nifi	login <code>admin</code> / <code>admin12345678</code>
Redpanda Console	http://localhost:8086	le broker est visible
Airflow	http://localhost:8090	mot de passe : <code>docker logs benefits-airflow grep "Password"</code>
Dremio	http://localhost:9047	création du compte admin au premier accès
Metabase	http://localhost:3002	assistant de configuration
Marquez	http://localhost:3000	UI vide (elle se remplira en S10)

3.4. Outillage local

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt      # dbt, elementary, openlineage-dbt, sqlfluff
pre-commit install                  # garde-fous locaux (S8)
cd dbt && dbt deps && dbt debug      # doit se terminer par « All checks passed! »
```

✅ **Checkpoint S3 :** `dbt debug` passe chez tout le monde ; chacun sait dire, pour trois conteneurs au choix, à quelle séquence des trois jours ils serviront.

S4 — Zone Bronze : ingestion, traçabilité et qualité technique

🎯 **Objectif** : comprendre comment les données passent de PostgreSQL au lakehouse **sans aucune transformation métier** mais avec une traçabilité systématique (RG_01), pourquoi il faut deux outils d'ingestion, et comment les trois RG de *qualité technique* (RG_30, 31, 32) forment le système d'alarme de l'ingestion.

🧰 **Outils** :

- **Apache NiFi** — ETL visuel par flux : idéal pour le **chargement initial batch** (toutes les 4 h), avec reprise sur erreur, backpressure et suivi visuel.
- **Debezium** — lit le Write-Ahead Log de PostgreSQL : chaque INSERT/UPDATE/DELETE devient un événement, sans requêter les tables (CDC).
- **Redpanda** — le bus d'événements compatible Kafka qui transporte ces événements (un topic par table).
- **Apache Iceberg + Nessie** — le format de table et le catalogue où tout atterrit, côté `bronze.*`.
- **dbt seed** — le raccourci d'atelier : les mêmes données, avec les mêmes colonnes de traçabilité, chargées en 10 secondes — exactement ce que fera la CI.
- **Elementary** — la bibliothèque d'observabilité qui apprend les volumes habituels (RG_31) et mémorise les schémas (RG_32).

4.1. Pourquoi deux outils d'ingestion ?

Besoin	Outil	Mode	Limite
Premier chargement complet	NiFi	Batch planifié (cron 4 h)	latence : jusqu'à 4 h de retard
Modifications au fil de l'eau	Debezium + Redpanda	Streaming CDC	complexité : WAL, offsets, tombstones

NiFi amorce le pipeline, Redpanda le maintient à jour. Les deux écrivent dans les **mêmes tables Bronze** et ajoutent les **mêmes colonnes RG_01** — dbt ne voit qu'une seule chose : les tables `bronze.*`. C'est aussi pour cela que la déduplication (RG_03) existe : les deux canaux *peuvent* livrer la même ligne.

4.2. RG_01 — la traçabilité d'ingestion

Colonne	Contenu	À quoi ça sert
<code>_ingested_at</code>	horodatage d'ingestion	fraîcheur (RG_02), déduplication (RG_03 : la plus récente gagne), anomalies de volume (RG_31)
<code>_source_system</code>	<code>postgres_src</code> ou <code>redpanda_cdc</code>	savoir quel canal a livré la ligne
<code>_batch_id</code>	UUID du lot	rejouer ou invalider un lot entier

4.3. Le flux NiFi (batch) — exercice guidé

Le flux complet est documenté dans [ingestion/nifi/flow_bronze.md](#) :

```
[QueryDatabaseTable] → [UpdateRecord] → [ConvertAvroToParquet] → [PutIceberg]
(PostgreSQL src)      (colonnes RG_01)      (MinIO/Nessie)
```

Reconstruire ce flux pour la table `individu` dans l'UI NiFi. Points d'attention à faire manipuler :

- `Maximum-value Columns = id` : c'est ce qui rend le chargement **incrémental** — NiFi mémorise le dernier `id` vu.
- Le processeur `UpdateRecord` injecte les trois colonnes RG_01 (Replacement Value Strategy = Literal Value).
- Débrancher volontairement la connexion PostgreSQL et observer la file d'attente : c'est la **backpressure**, le mécanisme qui évite de perdre des données quand l'aval ralentit.

4.4. Le flux CDC (streaming) — exercice guidé

```
# 1. Déclarer le connecteur Debezium (surveille le WAL de PostgreSQL)
curl -X POST http://localhost:8083/connectors \
  -H "Content-Type: application/json" \
  -d @ingestion/debezium/connector-postgres-benefits.json

# 2. Observer les topics dans la console Redpanda (http://localhost:8086)
#   → benefits.bronze.src.individu, benefits.bronze.src.prestation, ...

# 3. Provoquer un événement et le voir passer :
docker exec -it pocs-postgres psql -U user -d pocs \
  -c "update src.individu set email = 'nouveau@example.ch' where id = 1;"
#   (prérequis CDC : wal_level=logical sur la base source – vérifier avec
#   SHOW wal_level; sinon utiliser le service postgres du compose)

# 4. Le consommateur écrit dans Iceberg en ajoutant les colonnes RG_01 :
python ingestion/redpanda/consumer_bronze.py individu
```

Question à débattre au tableau : *que fait le pipeline si Debezium rejoue un événement déjà consommé ?* (Réponse : rien de grave — RG_03 dédupliquera en Silver. C'est le principe **at-least-once + déduplication aval**, plus robuste que de viser un exactly-once fragile.)

4.5. La zone Bronze vue par dbt : fraîcheur et volumétrie

Que les données arrivent par NiFi, Redpanda ou `dbt seed`, dbt les voit au même endroit : la source `bronze` de `dbt/models/staging/sources.yml`, qui porte **quatre lignes de défense** :

```
freshness:                                     # RG_02 – les données arrivent-elles encore ?
  warn_after: { count: 24, period: hour }
  error_after: { count: 72, period: hour }
tables:
  - name: prestation
    tests:
      - minimum_row_count:                     # RG_30 – la table est-elle anormalement vide ?
        entity: prestation                    #   (seuil lu dans le contrat)
      - elementary.schema_changes:            # RG_32 – le schéma source a-t-il dérivé ?
        config: { severity: warn }
```

et, côté modèle `stg_prestation` :

```
tests:
  - elementary.volume_anomalies:              # RG_31 – le volume de ce run est-il
    timestamp_column: _ingested_at            #   statistiquement normal ?
```

 **Concept — seuil fixe vs anomalie apprise.** RG_30 est un seuil **fixe** du contrat : simple, prévisible, mais à maintenir. RG_31 est **apprisé** : Elementary historise les volumes de chaque run et alerte sur l'écart statistique — aucun seuil à choisir, mais il lui faut de l'historique (elle est peu fiable les premiers jours). Les deux se complètent ; ne choisissez jamais « l'un ou l'autre ».

```
make seed                                     # charge les 4 CSV (zone Bronze simulée)
cd dbt && dbt source freshness                # RG_02
dbt test --select tag:rg30 tag:rg31 tag:rg32
```

 Si la fraîcheur alerte pendant l'atelier (les seeds datent), `make freshen` simule une ingestion fraîche — et c'est l'occasion de montrer ce que voit l'exploitation quand NiFi tombe.

 **Checkpoint S4 (fin du Jour 1)** : chacun a vu un événement CDC transiter dans la console Redpanda, sait expliquer *at-least-once + déduplication aval*, et sait dire laquelle des RG_02/30/31/32 se déclencherait si : (a) NiFi est arrêté depuis 3 jours, (b) la table prestation arrive vide, (c) une colonne `iban` apparaît dans la source.

JOUR 2 — Transformer et prouver

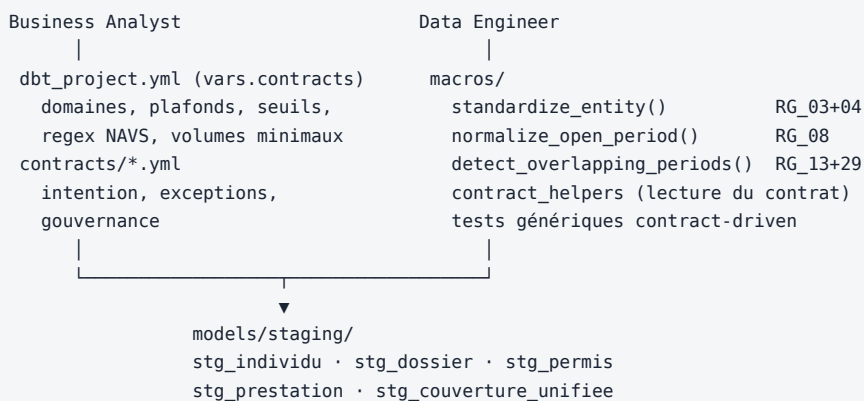
S5 — Zone Silver : le contrat à l'œuvre

🎯 **Objectif** : transformer le Bronze brut en données propres et standardisées en appliquant RG_03 à RG_13 et RG_28 — et constater que **pas une seule valeur métier n'est codée en dur dans le SQL** : tout vient du contrat. Comprendre aussi comment les PII sont maîtrisées dès cette zone.

🧰 **Outils** :

- **dbt Core** — le moteur de transformation : chaque modèle est un `SELECT`, dbt gère l'ordre, la matérialisation et les tests.
- **dbt-utils** — macros éprouvées : `deduplicate` (RG_03), `generate_surrogate_key` (RG_04).
- **Le contrat** (`vars.contracts`) — domaines, bornes et regex lus par les macros `dbt/macros/contract_helpers.sql`.
- **dbt snapshot** — l'historisation SCD2 (RG_11) sans écrire une ligne de logique temporelle.

5.1. L'architecture des responsabilités



5.2. Le moteur générique — une macro pour toutes les entités

`dbt/macros/standardize_entity.sql` applique RG_03 + RG_04 à n'importe quelle table Bronze :

```
{{ standardize_entity(
    source_relation = source('bronze', 'dossier'),
    partition_cols = ['numero_dossier', 'numero_individu', 'date_debut'],
    order_by = '_ingested_at desc',
    sk_cols = ['numero_dossier', 'numero_individu', 'date_debut']
) }}
```

Quatre modèles staging, une seule implémentation de la déduplication. Le jour où la stratégie change, on modifie **une** macro — et les unit tests de S7 le vérifient.

📖 **Concept — clé surrogate déterministe (RG_04)**. `generate_surrogate_key` produit un hash MD5 des colonnes métier : la même ligne donne la même clé en dev, en CI et en prod. C'est ce déterminisme qui permet de comparer des environnements entre eux et de faire des tests reproductibles — une séquence auto-incrémentée ne le permettrait pas.

5.3. Le contrat lu par le code — RG_09, RG_10, RG_15, RG_28

Regardez `dbt/models/staging/stg_prestation.sql` et `stg_individu.sql` : ni la liste des 9 types LASLP, ni les plafonds, ni la regex NAVS n'y figurent :

```
-- RG_10 : le domaine vient du contrat
({{ sql_in_contract_domain('type_prestation', 'prestation', 'domaine_type_prestation') }})
  as is_type_valide,

-- RG_15 : le CASE des plafonds est GÉNÉRÉ depuis le contrat
({{ flag_montant_suspect('type_prestation', 'montant_prestation') }}) as is_montant_suspect,

-- RG_28 : la regex NAVS13 vient du contrat
coalesce(navs ~ '{{ get_contract("individu", "navs_regex") }}', false) as is_navs_valide
```

Et le test générique `test_in_contract_domain.sql` rend le contrôle déclaratif dans `schema.yml` :

```
- name: permis
  tests:
    - in_contract_domain:
        entity: permis
        domain_key: domaine_permis
```

Exercice pivot des trois jours (30 min) : ajouter un plafond `FORFAIT_INTEGRATION: 1000` dans `vars.contracts.prestation.plafonds`, relancer `dbt build --select stg_prestation+`, vérifier avec `dbt compile` que le CASE généré contient la nouvelle branche, puis vérifier qu'une prestation des seeds passe au-dessus du nouveau plafond. *Aucun fichier SQL modifié*. Discuter : qu'aurait donné la même demande dans un pipeline « classique » ? (grep des valeurs en dur, N fichiers modifiés, risque d'en oublier un.)

5.4. PII et rôles — RG_07 (et la préparation de RG_33/34)

`stg_individu.sql` masque `nom`, `prenom`, `email`, `navs` sauf pour les rôles habilités du contrat :

```
dbt build --select stg_individu # analyste_bi → MASQUÉ
dbt build --select stg_individu --vars '{current_role: dpo}' # DPO → en clair
```

Trois subtilités à faire remarquer :

1. Le flag `is_navs_valide` (RG_28) est calculé **avant** le masquage : le contrôle qualité fonctionne même quand personne ne voit la valeur.
2. Chaque build est **journalisé** avec son rôle (RG_34, hook `on-run-end` — on le verra en S10) : le masquage sans journal ne satisferait pas un audit LPD.
3. La zone Gold n'aura **jamais** ces colonnes, même masquées (RG_33) : la minimisation, c'est ne pas transporter, pas seulement cacher.

5.5. La couverture unifiée — RG_08 + RG_12

`stg_couverture_unifiee` unionne dossiers et **permis valides** (un permis hors référentiel OCPM n'ouvre pas de droit) avec les périodes normalisées à 9999-12-31 (RG_08). Vérifier la couverture d'un versement (RG_18) = une seule jointure.

 **Concept — pourquoi 9999-12-31 et pas NULL ?** `date_prestation BETWEEN debut AND NULL` vaut NULL, donc *faux silencieusement* : chaque contrôle devrait répéter un `COALESCE`. Normaliser une fois en Silver (RG_08), c'est éliminer une classe entière de bugs en aval — au prix d'une convention qu'il faut documenter (elle l'est : dans le contrat métier `dossier.yml`).

5.6. L'historisation SCD2 — RG_11

```
make snapshot # dbt snapshot → snapshots.snap_permis_individu
```

Ivan Petrov (IND-009) passe de N à F le 15/03/2022 : après le snapshot, les deux versions coexistent avec `dbt_valid_from` / `dbt_valid_to`. Exercice : modifier un permis dans `bronze.permis` (UPDATE), relancer le snapshot, observer la nouvelle version, puis répondre : *quelle question d'audit ce mécanisme permet-il de traiter que la table source seule ne permet pas ?* (« Quel était son statut au moment du versement X ? »)

5.7. Construire la zone Silver

```
cd dbt
dbt build --select staging      # modèles + tests génériques
```

Les tests remontent des **warnings attendus** : les seeds contiennent des anomalies volontaires (permis `Z`, type `AIDE_INCONNUE`, montant `-50`, naissance en `2030`, NAVS malformé, permis en chevauchement). On les traite systématiquement en `S7`.

✅ **Checkpoint S5** : l'exercice du plafond fonctionne chez chacun ; le groupe sait citer les trois subtilités PII de 5.4 ; et personne ne sait donner un exemple de valeur métier codée en dur dans `models/staging/` (parce qu'il n'y en a pas).

S6 — Zone Gold : modélisation et RG métier avancées

🎯 **Objectif** : construire les tables prêtes pour la BI — dimension calendrier, table de faits, agrégats — en appliquant les RG les plus riches (RG_14 à RG_27), maîtriser deux pièges de modélisation (fan-out, dates de référence) et verrouiller le schéma de la table de faits par un contrat de colonnes.

🧰 **Outils** :

- `dbt_utils.date_spine` — l'axe temps (RG_21) sans table à maintenir.
- `dbt_utils.safe_divide` — le coût journalier sans division par zéro (RG_22).
- `dbt_utils.pivot` — une colonne par type de prestation... dont la liste **est le domaine du contrat**.
- `contract: enforced` — le contrat de **schéma** dbt : toute dérive de colonnes casse le build en CI. (À ne pas confondre avec le contrat de **données** `vars.contracts` : l'un fige la *forme*, l'autre pilote le *fond*.)

6.1. La table de faits — `fct_prestations`

Un versement par ligne, enrichi de **sept indicateurs de conformité** (`is_conforme_couverture`, `is_alerte_senior`, `is_alerte_lamal_age`, `is_verse_apres_deces`, `is_migrant_sans_permis`, `is_type_valide`, `is_montant_suspect`). Trois points de modélisation à traiter au tableau avant de lire le code :

RG_18 sans fan-out. Un individu peut avoir plusieurs couvertures simultanées (dossier + permis). Une jointure naïve dupliquerait les versements ; on utilise `EXISTS` :

```
exists (  
  select 1 from couverture c  
  where c.numero_individu = p.numero_individu  
        and p.date_prestation between c.date_debut and c.date_fin_normalisee  
) as is_conforme_couverture
```

Attribution du dossier sans doublon. Le rattachement au dossier actif à la date du versement passe par un `row_number()` — sinon le chevauchement volontaire de Gisèle Moreau (RG_13) dupliquerait ses lignes de faits. Principe général : **une RG violée en amont ne doit jamais casser la structure en aval** — elle doit rester *visible et comptée*.

Les RG « métier fin » : trois histoires, trois implémentations.

RG	Histoire métier	Implémentation
RG_25	« La LAMAL, c'est pour les jeunes adultes. »	âge à la date du versement (<code>age(date_prestation, date_naissance)</code>), bornes lues au contrat
RG_26	« On a continué à verser deux mois après le décès. »	<code>date_prestation > date_deces</code> — le flag le plus simple du projet et le plus grave en audit
RG_27	« Une aide d'urgence à quelqu'un qui n'a pas de permis ? »	<code>NOT EXISTS</code> sur les permis valides actifs, liste des types migrants lue au contrat

6.2. Les agrégats

- `agg_prestations_mensuelles` — grain individu × mois : total, nombre de versements, **RG_19** (plafond de cumul lu au contrat), et le **pivot** dont les colonnes `mnt_*` sont générées depuis le domaine du contrat. Ajouter un type de prestation au contrat = une colonne de plus au prochain build (et l'exercice de S8 le prouvera en CI).
- `agg_loyers_dossier_mensuel` — grain **dossier** × mois : **RG_17**. Rappel de S1 : le loyer se contrôle au foyer, pas à l'individu.

6.3. Le contrat de schéma — démonstration

```
dbt build --select marts
```

Puis casser volontairement : renommer `montant_par_jour` en `cout_journalier` dans `fct_prestations.sql` **sans** toucher au `schema.yml` → le build échoue immédiatement avec un diff de colonnes explicite. C'est ce qui arriverait en CI à quiconque modifie le schéma sans mettre à jour le contrat — et donc sans prévenir les consommateurs déclarés dans les expositions (S10). Annuler la modification.

6.4. Exercice — lire les anomalies comme un auditeur (20 min)

```
select numero_individu, type_prestation, date_prestation,  
       is_conforme_couverture, is_alerte_senior, is_alerte_lamal_age,  
       is_verse_apres_deces, is_migrant_sans_permis  
from gold.fct_prestations  
where not is_conforme_couverture or is_alerte_senior or is_alerte_lamal_age  
       or is_verse_apres_deces or is_migrant_sans_permis;
```

Pour chaque ligne remontée, le binôme rédige une phrase d'explication *métier* (pas technique) en se référant à S1. Corrigé : IND-999 inconnu du référentiel ; IND-010 permis expiré ; IND-007 senior ; IND-004 a 13 ans (LAMAL) ; IND-012 décédé le 15/02 ; IND-001 aide d'urgence sans permis.

✅ **Checkpoint S6** : le groupe explique chaque ligne de l'exercice 6.4 en langage métier, et sait dire pourquoi RG_25 utilise `date_prestation` et pas `current_date`.

S7 — La pyramide de tests et le cycle WAP

🎯 **Objectif** : maîtriser les **cinq étages de tests** du projet, dérouler le passage alerte → fail-fast, comprendre le « test des tests », et voir comment le cycle Write-Audit-Publish de Nessie permet de tester une évolution **sans jamais toucher la production**.

🛠️ **Outils** :

- **Unit tests dbt** (dbt ≥ 1.8) — testent la **logique** d'un modèle sur des données mockées, indépendamment des seeds : `dbt/models/marts/unit_tests.yml`.
- **Tests génériques** (schema.yml) — unicité, non-nullité, domaines contract-driven, bornes.
- **Tests singuliers** (dbt/tests/) — une requête SQL qui retourne les anomalies : 0 ligne = succès.
- **Elementary** — anomalies statistiques (RG_31/32) et rapport HTML de qualité.
- `dbt seed` — les 4 CSV piégés, une anomalie par RG à démontrer.
- **Nessie** — les branches Zero-Copy du lakehouse pour le WAP.

7.1. 📖 Concept — la pyramide de tests data

▲ « test des tests »	la CI vérifie que le mode strict DÉTECTE les seeds piégés
+ anomalies apprises	Elementary : volume, schéma (RG_31, 32)
+ tests singuliers	SQL libre : chevauchements, réconciliation (RG_13,16,23,24,29,33)
+ tests génériques	déclaratifs, contract-driven (RG_06,09,10,14,15,28,30...)
+ unit tests	logique pure sur données mockées (RG_18, 20, 22, 26)

Plus on descend, plus le feedback est rapide et localisé ; plus on monte, plus le test couvre large. **Un data test valide les données produites ; un unit test valide le code** — sur des cas que les seeds ne contiennent pas (et ne devraient pas contenir : on ne pollue pas un jeu de démonstration pour couvrir un cas limite de code).

7.2. Les unit tests dbt — nouveauté à manipuler

`unit_tests.yml` mocke les entrées de `fct_prestations` et fixe les sorties attendues :

- `rg22_cout_journalier_période_un_jour` — une prestation d'un seul jour → coût journalier = montant (pas de division par zéro) ;
- `rg18_couverture_active_et_inactive` — un versement dans la période → conforme ; hors période → non conforme ;
- `rg20_rg26_senior_et_deces` — 79 ans + forfait → alerte ; versement post-décès → flag.

```
make unit          # dbt test --select "test_type:unit" - s'exécute AVANT le build
```

Exercice : casser volontairement la macro `normalize_open_period` (remplacer 9999-12-31 par 2020-01-01), relancer `make unit` → le unit test RG_18 échoue **sans avoir buildé quoi que ce soit**. Restaurer. C'est la boucle de feedback la plus courte du projet après pre-commit.

7.3. Les anomalies volontaires des seeds

Seed	Cas piégé	RG déclenchée
<code>individu.csv</code>	IND-003 livré deux fois (batch puis CDC, emails différents)	RG_03 — la version récente gagne
<code>individu.csv</code>	IND-011 né en 2030, NAVS <code>NAVS-INVALIDE</code>	RG_06, RG_28
<code>individu.csv</code>	IND-007 née en 1952 ; IND-012 décédé le 15/02/2024	prépare RG_20, RG_26
<code>dossier.csv</code>	IND-007 deux fois sur DOS-2022-099, périodes ouvertes	RG_13
<code>permis.csv</code>	IND-005 : B→C ; IND-009 : N→F	RG_11 (SCD2)
<code>permis.csv</code>	IND-010 : permis <code>Z</code> ; IND-006 : B et L ouverts simultanément	RG_09, RG_29
<code>prestation.csv</code>	P-010 : 2° forfait entretien pour IND-001 en janvier	RG_16
<code>prestation.csv</code>	P-011 : 2° LOYER pour DOS-2020-001 en janvier	RG_17

Seed	Cas piégé	RG déclenchée
<code>prestation.csv</code>	P-012 : montant –50 CHF	RG_14
<code>prestation.csv</code>	P-013 : IND-999 inexistant	RG_23 (et RG_18)
<code>prestation.csv</code>	P-014 : LOYER à 3 200 CHF	RG_15
<code>prestation.csv</code>	P-015 : forfait entretien pour IND-007 (72 ans)	RG_20
<code>prestation.csv</code>	P-016 : type <code>AIDE_INCONNUE</code>	RG_10
<code>prestation.csv</code>	P-009 : versement à IND-010 sans couverture active	RG_18
<code>prestation.csv</code>	P-019 : PRIME_LAMAL pour IND-004 (13 ans)	RG_25
<code>prestation.csv</code>	P-020 : forfait pour IND-012, 3 semaines après son décès	RG_26
<code>prestation.csv</code>	P-021 : AIDE_URGENCE pour IND-001, qui n'a aucun permis	RG_27
<code>prestation.csv</code>	P-017/P-018 : deux AIDE_URGENCE le même mois	aucune — légitime
<code>prestation.csv</code>	P-007 : PRIME_LAMAL pour IND-003 (19 ans)	aucune — légitime
<code>prestation.csv</code>	IND-001 cumule 5 200 CHF en janvier	RG_19

Les deux cas « aucune » sont aussi importants que les autres : une suite de tests se juge autant sur ses **non-déclenchements** (faux positifs évités) que sur ses détections.

7.4. Dérouler la pyramide

```
make unit      # étage 1 : logique
make build     # étages 2-4 : tout construit, les anomalies = warnings
make test-strict # fail-fast : rg_severity=error → le pipeline S'ARRÊTE
make audit     # tableau de bord DPO : nb d'anomalies par RG
make report    # rapport Elementary (HTML) : historique, anomalies apprises
```

Résultat attendu de `make audit` (15 lignes) :

regle	nb_anomalies
RG_06 – Incohérence biographique	1
RG_09 – Permis hors référentiel OCPM	1
RG_10 – Type de prestation hors LASLP	1
RG_14 – Montant non strictement positif	1
RG_15 – Montant suspect (hors barème)	1
RG_16 – Forfait entretien multiple dans le mois	1
RG_17 – Doublet de LOYER par dossier	1
RG_18 – Versement sans couverture active	2
RG_19 – Dépassement de plafond mensuel	1
RG_20 – Forfait entretien versé à un senior	1
RG_25 – PRIME_LAMAL hors tranche 18-25 ans	1
RG_26 – Versement postérieur au décès	1
RG_27 – Prestation migrant sans permis actif	1
RG_28 – NAVS hors format NAVS13	1
RG_29 – Permis en chevauchement	1

7.5. Le cycle WAP avec Nessie

Sur le lakehouse, le même déroulé se fait **sur une branche de données** — aucune copie, aucun risque pour la production :

```
# WRITE : brancher le catalogue (Zero-Copy, instantané)
curl -X POST http://localhost:19120/api/v1/trees/branch \
  -H "Content-Type: application/json" \
  -d '{"name": "feature/rg17-loyer", "sourceRefName": "main"}'

# AUDIT : construire et tester sur la branche uniquement
# (cible dbt lakehouse : dbt-dremio/trino, branche passée en variable)
dbt build --vars '{nessie_branch: feature/rg17-loyer}'

# PUBLISH : zéro test en échec → merge vers main ; sinon on jette la branche
curl -X POST "http://localhost:19120/api/v1/trees/branch/feature%2Frg17-loyer/merge" \
  -H "Content-Type: application/json" \
  -d '{"fromRefName": "feature/rg17-loyer", "toRefName": "main"}'
```

Environnement	Branche Nessie	Rôle
Développement	feature/*	développer et tester une RG en isolation
Recette	main (Nessie)	validation métier
Production	référence promue	consommée par Dremio/Metabase

✅ **Checkpoint S7 (fin du Jour 2)** : chacun sait situer une règle donnée sur la pyramide, a vu `make test-strict` échouer et sait dire pourquoi c'est le comportement voulu, et a créé puis supprimé une branche Nessie.

JOUR 3 — Industrialiser

S8 — CI/CD : du poste du développeur à la production

🎯 **Objectif** : dérouler la chaîne complète des garde-fous — du hook pre-commit (secondes) au déploiement de la documentation (minutes) — et comprendre le « test des tests », le garde-fou le plus original du projet. Personne ne doit pouvoir fusionner du code — ou un changement de contrat — sans que la plateforme entière ait été reconstruite et testée.

🧰 Outils :

- **pre-commit** — `.pre-commit-config.yaml` : lint SQL, hygiène YAML et `dbt parse` avant le commit. La boucle de feedback la plus courte de la chaîne.
- **GitHub Actions** — `ci.yml` (à chaque PR) et `nightly.yml` (chaque nuit).
- **SQLFluff** — lint SQL/Jinja (`.sqlfluff`) : le contrôle le moins cher, donc le premier.
- **Service container PostgreSQL** — une base jetable par run : l'équivalent CI d'une branche Nessie Zero-Copy.
- **GitHub Pages** — la documentation dbt publiée automatiquement à chaque merge.

8.1. 📖 Concept — la chaîne des boucles de feedback

Garde-fou	Quand	Latence	Attrape
pre-commit	avant le commit	secondes	SQL mal formé, YAML invalide, projet qui ne parse plus
Unit tests	début de CI	< 1 min	régression de logique (RG_18, 20, 22, 26)
<code>dbt build</code> CI	à chaque PR	minutes	schéma cassé, test générique/singulier en échec
Test des tests	à chaque PR	minutes	RG neutralisée
Nightly	chaque nuit	heures	dérive des données (fraîcheur, volume, anomalies)

La règle d'or : chaque problème doit être attrapé par le garde-fou **le moins cher** capable de le voir.

8.2. Le pipeline de PR



8.3. Le « test des tests »

La CI vérifie que le mode strict **échoue bien** sur les anomalies volontaires des seeds :

```
- name: Contrôle fail-fast – le mode strict doit détecter les anomalies
run: |
  if dbt build --target ci --vars '{rg_severity: error}'; then
    echo "::error::Les RG bloquantes sont inopérantes."
    exit 1
  fi
```

Si quelqu'un neutralise une RG bloquante — en vidant un domaine du contrat, en supprimant un test, en désactivant une macro — la CI devient rouge **même si tous les autres jobs sont verts**. Les seeds piégés ne servent pas qu'à la démo : ils sont l'assurance-vie du dispositif. Question à débattre : *quelles attaques ce garde-fou n'attrape-t-il PAS ?* (une RG dont aucun seed ne déclenche l'anomalie — d'où la règle du mini-projet S12 : toute nouvelle RG arrive AVEC son seed piégé.)

8.4. Le nightly — valider les données, pas le code

`nightly.yml` tourne chaque nuit (et à la demande via `workflow_dispatch`) : fraîcheur RG_02, volumétrie RG_30, anomalies Elementary RG_31/32, rapport HTML en artefact. En production, ce rôle revient au DAG Airflow (S9) ; le nightly GitHub reste un filet **indépendant de l'orchestrateur** — si Airflow meurt, quelqu'un le saura quand même.

8.5. Exercices PR (30 min)

1. **PR verte** : ajouter un type `AIDE_TRANSPORT` au domaine du contrat → la CI passe, et le pivot de `agg_prestations_mensuelles` gagne une colonne `mnt_AIDE_TRANSPORT` (le vérifier dans les artefacts de docs).
2. **PR rouge : supprimer** `LOYER` du domaine → la CI échoue : les seeds contiennent des loyers devenus invalides.
Discussion : c'est le comportement attendu d'un contrat — on ne retire pas une promesse que des données existantes utilisent.
3. **PR sournoise (démo animateur)** : supprimer le test `accepted_values` de RG_14 en laissant tout le reste → jobs classiques verts, **test des tests rouge**.

✅ **Checkpoint S8** : le groupe sait dire, pour chacune des trois PR, quel étage de la chaîne l'a bloquée (ou laissée passer) et pourquoi.

S9 — Orchestration avec Airflow

🎯 **Objectif** : passer de « je lance `make build` à la main » à « la plateforme se reconstruit toute seule toutes les 4 heures, me réveille si ça casse, et rejoue uniquement ce qui a échoué ».

🧰 **Outils** :

- **Apache Airflow** — l'orchestrateur : DAGs (graphes de tâches), planification cron, retries, backfill, alerting. Ici en mode `standalone` (suffisant pour l'atelier).
- **Le DAG** `orchestration/airflow/dags/dbt_benefits_dag.py` — freshness → seed → build → snapshot → tests singuliers → docs, toutes les 4 h (la cadence NiFi).

9.1. 📖 Concept — qui orchestre quoi ?

Mécanisme	Déclencheur	Bon pour	Mauvais pour
cron NiFi	horloge	ingestion Bronze	dépendances entre étapes
GitHub Actions	événement Git / cron	valider le code , filet nightly	production data (pas fait pour)
Airflow	horloge + dépendances	le pipeline data de production : ordre, retries, reprise	remplacer la CI (il ne voit pas les PR)

Les trois coexistent, chacun à sa place. Le DAG dbt reflète le Makefile : ce qu'on a appris à lancer à la main, Airflow le lance à l'heure.

9.2. Démarrer et explorer

```
docker compose --profile orchestration up -d
docker logs benefits-airflow 2>&1 | grep "Password for user" # login admin
# → http://localhost:8090, DAG « benefits_lakehouse »
```

Anatomie du DAG (l'ouvrir dans l'UI, vue *Graph*) :

```
dbt_deps → source_freshness → dbt_seed → dbt_build → dbt_snapshot → dbt_test_singular → dbt_docs_generate
(RG_02 en première ligne : des sources périmées stoppent le run avant tout build)
```

9.3. Exercices (30 min)

1. **Déclencher** le DAG à la main (bouton ►) et suivre les logs de `dbt_build` en direct.
2. **Provoquer un échec** : arrêter `pocs-postgres` (`docker stop pocs-postgres`), redéclencher, observer le retry (1 tentative, 5 min) puis l'échec propre ; redémarrer la base, cliquer *Clear* sur la tâche échouée → Airflow **reprend là où il s'est arrêté**, sans rejouer ce qui a réussi. C'est l'argument décisif face à un cron.
3. Discuter : pourquoi la tâche `dbt_seed` disparaît-elle en production ? (NiFi/Redpanda alimentent Bronze ; le commentaire du DAG le dit explicitement.)

💡 En vrai projet, regarder **astronomer-cosmos** : il transforme chaque modèle dbt en tâche Airflow individuelle (reprise au modèle près, lignage Airflow par modèle). Ici, un opérateur par *étape* suffit et reste lisible.

✅ **Checkpoint S9** : chacun a déclenché le DAG, provoqué et réparé un échec, et sait expliquer la répartition cron NiFi / Actions / Airflow.

S10 — Lignage, documentation-as-code et outillage DPO

🎯 **Objectif** : rendre le travail **visible, navigable et auditable** : le lignage de bout en bout dans Marquez, la documentation générée depuis le code (et poussée jusque dans les commentaires PostgreSQL), les consommateurs déclarés (exposures), et les deux outils DPO : journal des accès PII (RG_34) et droit à l'oubli (RG_35).

🧰 **Outils** :

- **OpenLineage / Marquez** — `dbt-ol` intercepte chaque `dbt build` et pousse le graphe de lignage vers Marquez : qui produit quoi, à partir de quoi, quand, avec quel statut.
- **dbt docs** — la documentation vivante : doc blocks (`models/docs.md`), page d'accueil `__overview__`, graphe de dépendances.
- **persist_docs** — pousse les descriptions des `schema.yml` comme **commentaires SQL** dans PostgreSQL : la doc suit la donnée jusque dans pgAdmin et Dremio.
- **Exposures** — `models/marts/exposures.yml` : les consommateurs aval (dashboard Metabase, rapport DPO) déclarés comme du code.
- **Les macros DPO** — `macros/dpo_tools.sql` : RG_34 (hook) et RG_35 (run-operation).

10.1. Lignage automatique

```
export OPENLINEAGE_URL=http://localhost:5000
export OPENLINEAGE_NAMESPACE=benefits
make lineage          # dbt-ol build : chaque modèle émet ses événements
```

Ouvrir `http://localhost:3000` : le graphe complet `bronze.prestation` → `stg_prestation` → `fct_prestations` → `agg_prestations_mensuelles` apparaît, avec l'historique et le statut des runs. Exercices :

1. « Le DPO demande d'où vient `is_alerte_senior` » — remonter le graphe en 3 clics jusqu'à `bronze.individu.date_naissance`.
2. Relancer `make lineage` après avoir cassé un modèle : le run apparaît en échec **dans le lignage** — l'exploitation voit où la chaîne s'est arrêtée sans ouvrir un log dbt.

10.2. 📖 Concept — documentation-as-code, trois étages de sortie

La même source (`schema.yml` + `docs.md` + `contrats`) alimente **trois surfaces** sans double saisie :

Surface	Généré par	Public
Site dbt docs (<code>make docs</code> , et GitHub Pages à chaque merge)	<code>dbt docs generate</code>	équipe data, métier curieux
Commentaires SQL dans PostgreSQL (visibles pgAdmin/Dremio)	<code>persist_docs</code>	quiconque requête la base
Exposures dans le graphe (qui consomme quoi)	<code>exposures.yml</code>	gouvernance, analyses d'impact

```
make docs          # http://localhost:8087 - visiter : page d'accueil (__overview__),
                  # fct_prestations (descriptions par RG), onglet exposures
docker exec -it pocs-postgres psql -U user -d pocs \
  -c "\d+ gold.fct_prestations"      # ← les descriptions sont AUSSI là (persist_docs)
```

Exercice : `dbt ls --select +exposure:dashboard_pilotage_prestations` — la liste exacte de tout ce dont dépend le dashboard. C'est la réponse outillée à « peut-on supprimer ce modèle ? ».

10.3. Outillage DPO — RG_33, RG_34, RG_35 en action

RG_33 — minimisation. Le test `rg33_minimisation_pii_gold.sql` interroge `information_schema` : si une colonne `nom/email/navs` existe en zone Gold, il échoue — quel que soit son contenu. Démo : ajouter `p.numero_individu as email` dans un mart, builder, voir le test rouge, retirer.

RG_34 — journal des accès. Chaque run est tracé par le hook `on-run-end` :

```
select * from audit.pii_access_log order by executed_at desc limit 5;
-- invocation_id · commande · role_courant · pii_exposees · nb_noeuds
```

RG_35 — droit à l'oubli.

```
make forget IND=IND-011      # purge Bronze + rebuild (propagation Silver/Gold)
# psql : select * from audit.rgpd_forget_log; → la demande est journalisée SANS PII
make seed && make build      # restaurer les données d'atelier
```

Discussion : pourquoi purger **Bronze** et non chaque zone une à une ? (Bronze est la source de vérité du lakehouse : la purge se propage mécaniquement au build suivant — une purge zone par zone finirait incohérente.)

✅ **Checkpoint S10** : chacun a remonté le lignage d'une colonne, vu les descriptions dans `\d+`, listé les dépendances d'une exposure, et exécuté un oubli RGPD complet (purge → propagation → journal → restauration).

S11 — Restitution : Dremio et Metabase

 **Objectif** : exposer les données Gold aux utilisateurs finaux : vues haute performance dans Dremio, dashboard de pilotage et tableau d'alertes DPO dans Metabase.

 **Outils** :

- **Dremio** — moteur SQL du lakehouse : vues virtuelles (VDS) sur le catalogue Nessie, **Reflections** pour des réponses sub-secondes.
- **Metabase** — dashboards sans code pour le métier et le DPO.

11.1. Dremio (production lakehouse)

1. Connecter le catalogue Nessie (`http://nessie:19120`, bucket `warehouse`).
2. Créer des vues virtuelles sur `fct_prestations`, `agg_prestations_mensuelles`, `agg_loyers_dossier_mensuel`, `dim_calendrier`.
3. Activer une **Reflection** sur les vues d'agrégats et comparer le temps de réponse avant/après.

11.2. Metabase — dashboard « Pilotage des prestations »

En atelier, connecter directement PostgreSQL (`pocs` , schéma `gold`). Construire :

Carte	Contenu	RG
KPI du mois	Total versé, par type (colonnes pivotées <code>mnt_*</code>)	—
Taux de conformité	% de versements avec <code>is_conforme_couverture</code>	RG_18
Tableau des alertes DPO	montants suspects, plafonds dépassés, seniors, LAMAL hors âge, post-décès, migrants sans permis	RG_15, 19, 20, 25, 26, 27
Doublons de loyer	<code>agg_loyers_dossier_mensuel</code> où <code>is_doublon_loyer</code>	RG_17
Carte de chaleur	individu × mois × total mensuel	—

Remarque de gouvernance : ce dashboard est **déclaré** dans `exposures.yml` (S10) — si quelqu'un modifie `fct_prestations`, l'analyse d'impact (`dbt ls --select +exposure:...`) le liste immédiatement.

 **Checkpoint S11** : le tableau des alertes affiche les 6 familles d'anomalies des seeds, et chacun sait retrouver l'exposure correspondante dans dbt docs.

S12 — Mini-projet fil rouge : votre RG_36 de bout en bout

🎯 **Objectif** : consolider les trois jours en implémentant, par binôme, **une nouvelle règle de gestion complète** — du contrat au dashboard, en passant par le test, le seed piégé et la PR. C'est l'évaluation pratique de l'atelier.

🛠️ **Outils** : tous ceux des trois jours. L'animateur n'aide que sur les blocages d'environnement.

12.1. Sujets au choix (un par binôme)

Sujet	Énoncé métier	Difficulté
A — Rétention	« Les prestations de plus de 10 ans ne doivent plus apparaître en Gold (archivage légal). » Durée au contrat.	★★
B — AVANCE_RENTE orpheline	« Une AVANCE_RENTE sans aucun autre versement dans les 6 mois suivants est suspecte (la rente est censée arriver). »	★★★
C — Dossier fantôme	« Un dossier dont tous les rattachements sont clos mais qui reçoit encore un LOYER est une anomalie. »	★★★
D — Sujet libre	Une des questions collectées en S1 (validée par l'animateur).	★→★★★★

12.2. Étapes imposées (le chemin est le livrable)

1. **Contrat métier** : ajouter la règle dans `contracts/*.yml` (intention, gravité, exceptions).
2. **Contrat de données** : si la règle a un paramètre (durée, seuil...), l'ajouter dans `vars.contracts`.
3. **Implémentation** : flag dans un modèle, ou test singulier — justifier le choix.
4. **Seed piégé** : ajouter la ligne qui déclenche la règle **et** une ligne légitime voisine qui ne la déclenche pas.
5. **Test** : générique (`accepted_values` sur le flag) ou singulier, sévérité pilotée par `rg_severity` si bloquante.
6. **Doc** : description de colonne (elle apparaîtra dans dbt docs et en commentaire PostgreSQL), mention dans l'audit DPO si pertinent.
7. **PR** : ouvrir la Pull Request, la CI doit être verte **y compris le test des tests** (qui prouve que votre seed piégé déclenche bien votre règle en mode strict).
8. **Restitution** (5 min/binôme) : montrer le diff, le test qui détecte, la carte Metabase ou la ligne d'audit.

12.3. Grille d'évaluation

Critère	Points
La règle est paramétrée par le contrat (zéro valeur en dur)	/4
Le seed piégé déclenche ; le cas légitime voisin ne déclenche pas	/4
Sévérité justifiée (bloquant vs alerte) en termes métier	/3
Documentation : intention dans le contrat + description de colonne	/3
CI verte, test des tests compris	/3
Clarté de la restitution (langage métier)	/3
Total	/20

Corrigé indicatif animateur (sujet A) : `retention_annees: 10` dans `vars.contracts.prestation` ; puis débat filtre vs flag — filtrer fait « disparaître » des lignes (attention à RG_24, la réconciliation !), flaguer les garde visibles ; la bonne réponse dépend de l'exigence légale, et c'est exactement le genre d'arbitrage que l'atelier veut apprendre à verbaliser.

Synthèse des 35 règles de gestion

RG	Zone	Nom	Mécanisme	Fichier	Gravité
RG_01	Bronze	Traçabilité d'ingestion	<code>_ingested_at</code> / <code>_source_system</code> / <code>_batch_id</code>	ingestion/* + seeds	Obligatoire
RG_02	Bronze	Fraîcheur des sources	<code>dbt source freshness</code>	staging/sources.yml	Alerte→Bloquant
RG_03	Silver	Déduplication technique	<code>dbt_utils.deduplicate</code>	macros/standardize_entity.sql	Bloquant
RG_04	Silver	Clé surrogate déterministe	<code>dbt_utils.generate_surrogate_key</code>	macros/standardize_entity.sql	Obligatoire
RG_05	Silver	Normalisation sexe	<code>CASE WHEN</code>	staging/stg_individu.sql	Informatif
RG_06	Silver	Cohérence biographique	flag + <code>accepted_values</code> (borne au contrat)	staging/stg_individu.sql	Bloquant
RG_07	Silver	Anonymisation PII	Jinja + <code>var(current_role)</code>	staging/stg_individu.sql	Bloquant
RG_08	Silver	Périodes ouvertes	<code>coalesce</code> → 9999-12-31	macros/normalize_open_period.sql	Obligatoire
RG_09	Silver	Domaine permis OCPM	test générique <code>in_contract_domain</code>	contrat + macros/tests	Bloquant
RG_10	Silver	Domaine type prestation	test générique <code>in_contract_domain</code>	contrat + macros/tests	Bloquant
RG_11	Silver	Historisation SCD2 permis	<code>dbt_snapshot</code> (strategy check)	snapshots/	Obligatoire
RG_12	Silver	Union couverture	<code>UNION ALL</code> (permis valides seulement)	staging/stg_couverture_unifiee.sql	Obligatoire
RG_13	Silver	Non-chevauchement dossiers	macro self-join + test singulier	tests/rg13_*.sql	Bloquant
RG_14	Gold	Montant strictement positif	<code>dbt_expectations</code> between strict	staging/schema.yml	Fail-fast
RG_15	Gold	Plausibilité montant	CASE généré depuis les plafonds du contrat	macros/contract_helpers.sql	Alerte
RG_16	Gold	Unicité forfait mensuel	test singulier <code>GROUP BY HAVING</code>	tests/rg16_*.sql	Bloquant
RG_17	Gold	Unicité LOYER par dossier	agrégat dossier×mois + flag	marts/agg_loyers_dossier_mensuel.sql	Bloquant
RG_18	Gold	Conformité couverture	<code>EXISTS</code> + unit test	marts/fct_prestations.sql	Alerte
RG_19	Gold	Plafond cumul mensuel	<code>SUM ></code> seuil du contrat	marts/agg_prestations_mensuelles.sql	Alerte
RG_20	Gold	Senior + forfait entretien	âge ≥ seuil du contrat + unit test	marts/fct_prestations.sql	Alerte
RG_21	Gold	Dimension calendrier	<code>dbt_utils.date_spine</code>	marts/dim_calendrier.sql	Obligatoire
RG_22	Gold	Coût journalier sécurisé	<code>safe_divide</code> + <code>GREATEST</code> + unit test	marts/fct_prestations.sql	Obligatoire
RG_23	Transverse	Intégrité référentielle	test singulier anti-orphelins	tests/rg23_*.sql	Bloquant
RG_24	Transverse	Réconciliation Bronze→Gold	test singulier, tolérance 10 %	tests/rg24_*.sql	Bloquant

RG	Zone	Nom	Mécanisme	Fichier	Gravité
RG_25	Gold	Âge PRIME_LAMAL	âge à la date du versement, bornes au contrat	marts/fct_prestations.sql	Alerte
RG_26	Gold	Versement après décès	<code>date_prestation > date_deces</code> + unit test	marts/fct_prestations.sql	Bloquant
RG_27	Gold	Migrant sans permis	<code>NOT EXISTS</code> permis actifs, types au contrat	marts/fct_prestations.sql	Alerte
RG_28	Silver	Format NAVS13	regex du contrat, calculée avant masquage	staging/stg_individu.sql	Bloquant
RG_29	Transverse	Non-chevauchement permis	même macro que RG_13	tests/rg29_*.sql	Bloquant
RG_30	Bronze	Volumétrie minimale	test générique <code>minimum_row_count</code> (seuils au contrat)	macros/tests + sources.yml	Bloquant
RG_31	Bronze	Anomalies de volume	<code>elementary.volume_anomalies</code> (apprentissage)	staging/schema.yml	Alerte
RG_32	Bronze	Dérive de schéma source	<code>elementary.schema_changes</code>	staging/sources.yml	Alerte
RG_33	Conformité	Minimisation PII en Gold	test singulier sur <code>information_schema</code>	tests/rg33_*.sql	Bloquant
RG_34	Conformité	Journal des accès PII	hook <code>on-run-end</code> → <code>audit.pii_access_log</code>	macros/dpo_tools.sql	Obligatoire
RG_35	Conformité	Droit à l'oubli	<code>run-operation rgpd_forget</code> + journal	macros/dpo_tools.sql	Obligatoire

Quiz de clôture (15 min)

1. Le métier veut porter le plafond LOYER à 2 800 CHF : quels fichiers changent ? (un seul : le contrat dans `dbt_project.yml`)
2. Pourquoi RG_17 se teste au grain dossier et RG_16 au grain individu ?
3. Que se passe-t-il si un développeur supprime le test RG_14 ? (le « test des tests » de la CI devient rouge)
4. Deux AIDE_URGENCE le même mois : anomalie ou pas ? Où est-ce documenté ? (légitime — `contracts/prestation.yml`, *exceptions connues*)
5. Différence entre le unit test RG_18 et le test générique RG_18 ? (le premier valide la logique sur des mocks, le second les données réelles)
6. Pourquoi RG_25 évalue l'âge à `date_prestation` et pas à `current_date` ?
7. RG_30 et RG_31 détectent tous deux un problème de volume : lequel choisir ? (les deux — seuil fixe contractuel + anomalie apprise)
8. Le DPO demande « qui a vu des PII le mois dernier ? » : quelle table, quelle RG ? (`audit.pii_access_log`, RG_34)
9. Après un `rgpd_forget`, l'individu apparaît-il encore en Gold ? (oui, jusqu'au prochain build — c'est pour ça que la macro le rappelle)
10. Airflow, GitHub Actions, cron NiFi : qui valide le code, qui produit la donnée, qui ingère ?

Annexes

A. Démarrage rapide (rappel)

```
cp .env.example .env
docker compose --profile "*" up -d
python -m venv .venv && source .venv/bin/activate && pip install -r requirements.txt
pre-commit install
make seed && make unit && make build && make audit
```

B. Dépannage

Symptôme	Cause probable	Remède
<code>dbt debug</code> échoue	postgres pas prêt ou <code>.env</code> non chargé	<code>docker compose ps</code> , exporter les <code>DBT_*</code>
Dremio ne démarre pas	mémoire insuffisante	démarrer sans le profil <code>bi</code> , revenir en S11
Airflow long à démarrer	installation pip de dbt au premier boot	attendre 2-3 min ; <code>docker logs -f benefits-airflow</code>
Mot de passe Airflow introuvable	mode standalone	<code>docker logs benefits-airflow grep "Password for user"</code>
Le DAG ne voit pas la base	résolution <code>host.docker.internal</code>	vérifier <code>extra_hosts</code> (host-gateway) dans le compose
Marquez UI vide	<code>OPENLINEAGE_URL</code> non exporté	exporter puis relancer <code>make lineage</code>
<code>dbt source freshness</code> en erreur	seeds anciens	<code>make freshen</code>
RG_31 (Elementary) alerte bizarrement	pas assez d'historique de runs	normal les premiers jours — en discuter, c'est le concept
Port 3000/5432 occupé	service local existant	adapter les ports côté hôte dans docker-compose.yml
Unit test échoue sur la précision numérique	comparaison exacte	caster/arrondir dans le modèle, attendu en chaîne (<code>"100.00"</code>)
La CI échoue sur « fail-fast » en étant verte localement	une RG bloquante a été neutralisée	comparer contrat et tests avec <code>git diff main</code>

C. Pour aller plus loin

- Brancher dbt sur le lakehouse réel (dbt-dremio ou Trino + Nessie) et rejouer S5-S7 avec le WAP complet.
- Remplacer les BashOperators du DAG par **astronomer-cosmos** (une tâche Airflow par modèle dbt).
- Ajouter le **column-level lineage** (OpenLineage le supporte sur certains adapters) et comparer avec le lignage modèle.
- Étendre RG_31 : anomalies Elementary sur les *dimensions* (répartition par type de prestation) et non plus seulement le volume.
- Industrialiser le droit à l'oubli : file de demandes, SLA de purge, preuve d'exécution signée.